Class 6: R Functions Lab

Hubert Phan

Table of contents

Gradebook Function 1
Add function 6
DNA Sequence Generator 7
Protein Function 8

Gradebook Function

> Q1. Write a function grade() to determine an overall grade from a vector of student homework assignment scores dropping the lowest single score. If a student misses a homework (i.e. has anNA value) this can be used as a score to be potentially dropped. Your final function should beadquately explained with code comments and be able to work on an example class gradebook such as this one in CSV format: "https://tinyurl.com/gradeinput" [3pts]

```
library(dplyr)
library(tidyr)


#| label: Define grade function
#| warning: false
#| message: false

#' Grade Function: Determine overall grades after dropping
#' lowest score from a vector of homework scores. Missing values
#' will be treated as 0.
#'
#' @param scores. A data frame where each row represents a
#' student and each column represents a homework assignment score.
```

```
#'
#' @returns A list containing: A named vector of final grades
#' for each student after dropping the lowest score,
#' and a data frame with difficulty metrics for each assignment.
#' - grades: A named vector where names are student
#' identifiers (row names of the input
#' data frame) and values are the calculated final grades.
#' - difficulty_analysis: A data frame with two metrics
#' for each assignment:
#'   - true_difficulty: The median score of all submissions
#'   (excluding NAs).
#'   - submission_rate: The proportion of students who
#'   submitted the assignment (non-NA values).
#' @export
#'
#' @examples
#' gradebook <- data.frame("StudentID" = c("S1", "S2", "S3"),
#' Homework1 = c(85, 90, NA), Homework2 = c(80, NA, 70),
#' Homework3 = c(90, 95, 85))
#' grade(gradebook[,-1]) # Exclude StudentID column for grading
#'
grade <- function(scores) {#assume that scores is df
  #grab hw_cols
  #no need to grab id_col because we will set
  #row.names = 1 when reading in the data
  hw_cols <- names(scores)
  #ANALYZE difficulty: need to handle NA values, need to
  #drop lowest score per student, need to calculate
  #average of remaining scores
  #cant just replace NA values with 0 because that
  #would unfairly lower the average
  #treat 0 as a valid low score, but NA as a non-submission
  #consider median scores: The median is a better indicator
  #of difficulty than the mean because it is not influenced
  #by extreme values

  #consider a ratio of scores and actual attempts. So difficultly rating = median(score)/num
      #High Difficulty (Low Scores) + High Submission: A genuinely hard assignment.
      #Low Difficulty (High Scores) + Low Submission: An assignment that wasn't hard to pass
      #low Difficulty (high Scores) + high Submission: An easy assignment that most students
      #High Difficulty (Low Scores) + Low Submission: the assignment was hard and many peopl
```

```
  #create named vectors to store true difficulty and submission rate to make downstream analy
  true_difficulty <- sapply(scores[hw_cols], median, na.rm = TRUE)
  submission_rate <- sapply(scores[hw_cols], function(x) mean(!is.na(x)))


  #now handle NA values in scores by replacing them with 0 for the purpose of dropping the lo
  scores_clean <- scores %>%
    mutate(across(all_of(hw_cols), ~ replace_na(., 0)))
  #determine the grade per student after removing the lowest score ((sum of all assignments-l
  results <- scores_clean %>%
    rowwise() %>%
    mutate(
      #find minimum score per student and sum
      row_min   = min(c_across(all_of(hw_cols))),
      row_sum   = sum(c_across(all_of(hw_cols))),

      #calculate adjusted sum and count
      #do not double subtract, do not subtract char value
      final_grade = (row_sum - row_min) / (length(hw_cols) - 1)
      #calculate final grade, this appends a new column to the dataframe
    ) %>%
    ungroup()

  #Create named vector of grades
  final_grades <- setNames(results$final_grade, rownames(scores))


  return(list(
    grades = final_grades,
    true_difficulty_Medians = true_difficulty,
    submission_rate = submission_rate
  ))
}
```

Q2. Using your grade() function and the supplied gradebook, Who is the top scoring student overall in the gradebook? [3pts]

```
# Load the gradebook data from the provided URL
url <- "https://tinyurl.com/gradeinput"
gradebook <- read.csv(url, row.names = 1)
head(gradebook)
```

```
          hw1 hw2 hw3 hw4 hw5
student-1 100  73 100  88  79
student-2  85  64  78  89  78
student-3  83  69  77 100  77
student-4  88  NA  73 100  76
student-5  88 100  75  86  79
student-6  89  78 100  89  77
```

Now apply the grade function to the gradebook and determine the top scoring student.

```
# Apply the grade function to the gradebook
results <- grade(gradebook)
# Extract the grades
grades <- results$grades
# Identify the top scoring student
top_student <- (which.max(grades))
top_student
```

```
student-18
        18
```

Q3. From your analysis of the gradebook, which homework was toughest on students (i.e. obtained the lowest scores overall? [2pts]

```
# Extract difficulty analysis from the results
difficulty_results <- results$true_difficulty_Medians
# Identify the toughest homework based on true difficulty
toughest_homework <- which.min(difficulty_results)
toughest_homework
```

```
hw2
  2
```

Lets create a bar plot to visualize the difficulty of each homework assignment and confirm our findings.

```
boxplot(gradebook,
        main = "Homework Difficulty Analysis",
        xlab = "Homework Assignments",
        ylab = "Scores")
```

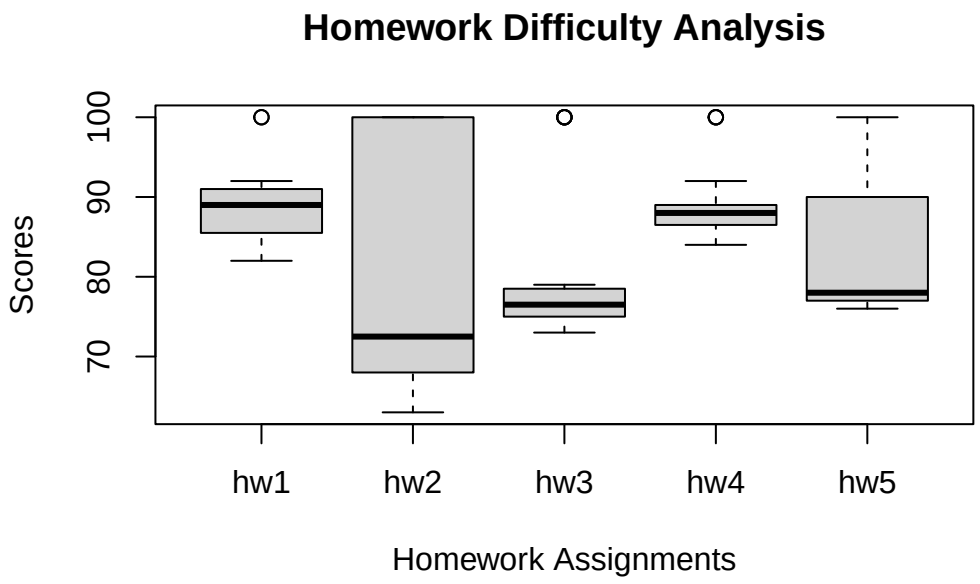

Figure 1: Homework Difficulty Analysis

So we can confirm that homework 2 was the toughest on students as it has the lowest median score among all assignments, despite having outliers with high scores.

> Q4. Optional Extension: From your analysis of the gradebook, which homework was most predictive of overall score (i.e. highest correlation with average grade score)? [1pt]

```
# The question asks are the final results for each student correlated with each homework assi
hw_cols <- names(gradebook) #homework columns
final_grades <- results$grades #student grades, with drop applied

correlations <- sapply(hw_cols, function(hw) { #apply a function over each homework column
  #x = scores on this homework, y = final grades
  cor(gradebook[[hw]], final_grades, use = "complete.obs")
})
correlations
```

```
       hw1         hw2         hw3         hw4         hw5
0.42502036  0.61142768  0.30425610 -0.09644108  0.60398041
```

```r
# Identify the most predictive homework based on highest correlation
most_predictive_homework <- which.max(correlations)
most_predictive_homework
```

```
hw2
  2
```

Add function

Write your first R function, add() to add some numbers

```r
#' Add Function: Adds two numbers together.
add <- function(x, y) {
  return(x + y)
}
```

Simple Example:

```r
add(2, 3) # Should return 5
```

```
[1] 5
```

Example where inputs are vectors:

```r
add(c(1, 2), c(3, 4)) # Should return c(4, 6)
```

```
[1] 4 6
```

Example where inputs are unequal length vectors:

```r
add(c(1, 2, 3), c(4, 5)) # Should return c(5, 7, 7) due to recycling
```

```
Warning in x + y: longer object length is not a multiple of shorter object
length
```

```
[1] 5 7 7
```

Example where three inputs are provided:

```
#add(1, 2, 3) # returns an error because add() only accepts two arguments
```

DNA Sequence Generator

> Q. Write a second function, generate_dna() to return nucleotide sequences of a user specified length.

```
#' Generate DNA Function: Generates a random DNA sequence of specified length.

generate_dna <- function(length) {
  # Define the nucleotides
  nucleotides <- c("A", "T", "C", "G")

  # Sample nucleotides with replacement to create a sequence
  dna_sequence <- sample(nucleotides, size = length, replace = TRUE)
}

# Example usage:
ex1 <- generate_dna(10) # Should return a random DNA sequence of length 10
ex1
```

```
 [1] "A" "C" "C" "G" "T" "A" "C" "G" "T" "T"
```

Improve the function to return the sequence as a single string rather than a vector of characters.

```
#' Improved Generate DNA Function: Generates a random DNA sequence of specified length as a s

generate_dna <- function(length) {
  # Define the nucleotides
  nucleotides <- c("A", "T", "C", "G")

  # Sample nucleotides with replacement to create a sequence
  dna_sequence <- sample(nucleotides, size = length, replace = TRUE)

  # Collapse the vector into a single string
  dna_string <- paste(dna_sequence, collapse = "")

  return(dna_string)
}
```

```r
# Example usage:
ex2 <- generate_dna(10) # Should return a random DNA sequence of length
ex2
```

```
[1] "TCTTAAGGGC"
```

Protein Function

> Q. Write a third function, generate_protein() to return protein sequences of different lengths and test whether these sequences are unique in nature.

```r
#' Generate Protein Function: Generates a random protein sequence of specified length as a si
#' Tests with BLAST to see if protein seq exists.

generate_protein <- function(length) {
  # Define the amino acids
  amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",
                   "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")

  # Sample amino acids with replacement to create a sequence
  protein_sequence <- sample(amino_acids, size = length, replace = TRUE)

  # Collapse the vector into a single string
  protein_string <- paste(protein_sequence, collapse = "")

  return(protein_string)
}

#create protein sequences of lengths from 6-12 in FASTA formate
protein_seqs <- sapply(6:12, generate_protein)
names(protein_seqs) <- paste0(">", 6:12)
fasta_output <- paste(names(protein_seqs), protein_seqs, sep = "\n")
cat(fasta_output, sep = "\n")
```

```
>6
SLQRPC
>7
VARRYLH
>8
NMTYWFSL
```

```
>9
MVQLRIRYD
>10
YSYFMWIMVT
>11
KPKDGPSPSEC
>12
HKYYDWMIEDVS
```

Q. Which sequences are unique?

The sequences HMETFW, WEGYAGI, and HRFGTKTA were not unique as they are found in nature with 100% identity and 100% query coverage according to BLAST results. The other sequences generated were unique and not found in nature based on BLAST search results.